

C++ 전처리 파이프라인 단계별 시각화 + 복습 가이드

아동 인물화 원본 이미지가 C++ 전처리 서버를 통과하면서 어떻게 변하는지 확인합니다. 각 단계는 다음 두 섹션으로 구성됩니다:

- **Why:** 왜 이 처리가 필요한지 + 다른 후보값을 안 고른 이유 + ADR 실측 수치
- **직접 테스트:** 코드를 실제로 변경해 결과 차이를 눈으로 확인하는 실험 + 내 관찰 결과 기록란

이미지 생성 방법은 [README.md](#) 참조 정량 벤치마크 원본: [docs/standards/ADR-parameter-rationale.md](#) 학습 방법론 원본: [docs/learning/preprocess_server_learning_guide.md](#)

파이프라인 개요

원본 (임의 크기)

- |
- ▼ [1] ResizeFilter(768)
- ▼ [2] ColorValidationFilter ← 컬러 이미지면 422 즉시 반환
- ▼ [3] DenoiseFilter
- ▼ [4] HybridPreprocessFilter
 - ├ 4a. Grayscale 변환
 - ├ 4b. 적응형 이진화 (AdaptiveThreshold)
 - ├ 4c. 형태학 연산 (MORPH_CLOSE)
 - ├ 4d. Smart Crop (ROI 감지)
 - ├ 4e. Letterbox Resize (512×512)
 - └ 4f. 3채널 병합 (R/G/B)
- |
- ▼ 512×512×3 RGB 이미지 → Python AI 서버

코드 위치: [preprocess-server/src/core/pipeline_factory.cpp](#)

Stage 00: 원본 입력

이미지

이미지



- 형식: BGR, 임의 크기
- 특징: 사용자가 업로드한 그대로. 해상도, 비율, 밝기 모두 불규칙

Stage 01: ResizeFilter (장축 768)

입력

출력

입력

출력



- **파라미터:** `targetSize=768, withPadding=false`
- **동작:** 장축이 768이 되도록 비율 유지 리사이즈 (패딩 없음). 보간법은 `INTER_AREA`

[01] Why — 왜 장축 768인가?

이후 모든 픽셀 연산(이진화, 형태학, 거리 변환)을 고정 해상도에서 실행해 레이턴시를 줄이는 단계. 실측: 원본 크기 유지 시 183ms → 768 리사이즈 후 97ms = **47% 감소**.

다른 후보를 안 고른 이유:

후보	문제점
512 (최종 출력과 동일)	Smart Crop 이후 ROI 영역이 더 작아짐 → Letterbox 시 upscaling 발생 → 화질 열화
1024	이진화·형태학 연산 픽셀 수 ≈ 4 배 ($1024^2 / 512^2 = 4$) → 레이턴시 급증
768	512×1.5 → Smart Crop에 충분한 해상도 헤드룸 + 연산 비용 균형

왜 `INTER_AREA`인가?:

- 축소 시 픽셀 영역 평균화로 모아레/aliasing 방지 (교재 §4 보간법 비교)
- `INTER_LINEAR`는 일반 리사이즈에 무난하지만 이진 마스크 축소 시 에지에 회색 번짐 발생
- 코드: `resize_filter.cpp`

[01] 직접 테스트

1. 동일 입력에 `targetSize`를 512 / 768 / 1024로 바꿔가며 출력 → 처리 시간(ms)을 로그로 비교
2. `INTER_AREA`를 `INTER_LINEAR`로 바꾼 뒤 결과를 `imwrite`로 저장 → 확대해서 에지 번짐 관찰

관찰 포인트:

- 512: Smart Crop 이후 디테일이 어떻게 변하는가?
- 1024: 전체 처리 시간이 얼마나 늘어나는가?
- INTER_LINEAR: 이진화 결과의 에지가 부드러워지는가?

🔍 내 관찰 결과:

- 512: 처리 시간은 가장 안정적이지만 선 디테일이 가장 많이 압축되어 연필선처럼 명암이 약한 입력에서는 Smart Crop으로 인물 영역을 맞춘 뒤에도 작은 선, 눈/입 주변, 손가락처럼 좁은 획이 뭉치거나 약해질 가능성이 큼
- 1024: INTER_AREA는 약 55% 증가, INTER_LINEAR는 약 83% 증가. 픽셀 수 기준으로 1024x1024는 768x768보다 약 1.78배 많아 실제 전체 파이프라인에서는 리사이즈뿐 아니라 후속 필터까지 포함하면 1024가 768보다 체감 비용이 꽤 커질 수 있음
- INTER_LINEAR: 이진 마스크에 INTER_LINEAR를 쓰면 에지가 부드러워질 가능성이 높고, 0/255 사이의 중간 회색값이 생길 수 있습니다. 이진 마스크 관점에서는 장점이 아니라 위험임. 이진성이 깨지면 이후 morphology, contour, distance transform, ROI 판단에 미세한 영향을 줄 수 있음.

보간법	768 처리 시간(ms)	1024 처리 시간(ms)	증가 시간(ms)	1024/768 비율	증가율
---	---	---	---	---	---
INTER_AREA	11.0433	17.1010	+6.0577	약 1.55배	약 +54.9%
INTER_LINEAR	3.3276	6.0902	+2.7626	약 1.83배	약 +83.0%

관련 문서: [ADR-parameter-rationale.md](#)

Stage 02: DenoiseFilter

입력	출력
	

- **파라미터** (ADR 권장): gaussianSize=3, medianSize=0 (비활성화)
- **동작**: Gaussian Blur로 고주파 노이즈 제거

△ 시각화 파이프라인이 `gaussianSize=5`로 표기되어 있다면, ADR 실측 권장값(3)으로 통일 필요.

[02] Why — 왜 Gaussian 3, 그리고 Median은 왜 비활성화인가?

이진화(AdaptiveThreshold)는 노이즈에 민감해서, 사전에 스무딩하지 않으면 연필 질감의 미세한 텍스처가 선(stroke)으로 오인되어 불필요한 점들이 생긴다.

ADR 실측 (입력: 실제 아동 인물화):

커널 크기	PSNR (dB)	SSIM	판정
3	36.3	0.9883	선택: 최고 SSIM, 노이즈 제거 최소
5	33.6	0.9765	중간 블러링
7	31.6	0.9605	과도한 블러 → 얇은 연필선 손실

- 아동 인물화의 선은 보통 1~2px → 커널이 커질수록 평균화로 선이 흐려짐
- **Median Blur 비활성화 이유:** Median은 고립 노이즈 제거에는 강하지만, 인물화의 얇은 선을 점으로 인식해 **선 자체를 제거**할 수 있음
- 코드: [denoise_filter.cpp](#)

[02] 직접 테스트

1. `gaussianSize`를 3 / 5 / 7로 바꿔가며 결과 저장
2. 다음 단계(이진화) 결과를 비교 → 미세 점이 늘어나는지/얇은 선이 사라지는지 관찰
3. `gaussianSize=6` (짝수) 시도 → OpenCV 에러 "ksize must be odd" 확인 (교재 §3 가우시안 커널 홀수 제약 체감)
4. Median Blur 활성화(`medianSize=3`) 후 얇은 선 변화 관찰

📖 내 관찰 결과:

- 커널 3 → 5 → 7로 갈수록:
 - denoise 결과에서 연필선 주변이 점점 부드러워지고, `gaussianSize=7`에서는 얇은 선 경계가 더 넓게 퍼져 보임.
 - 다음 단계 이진화 결과에서는 작은 컴포넌트 수가 12 → 3 → 1로 감소했지만, 이진 픽셀 수는 14031 → 15188 → 16833으로 증가함.
 - 즉 강한 Gaussian은 미세 점을 줄이는 동시에 선을 병합/확장할 수 있어, 얇은 획 보존 관점에서는 `gaussianSize=3`이 가장 보수적임.
- Median 활성화 시 선의 변화:
 - `gaussianSize=3`, `medianSize=3`에서는 작은 컴포넌트 수가 12 → 4로 줄어듦.
 - 이번 샘플의 주요 선은 사라지지 않았지만, edge crop 기준으로 선 질감이 더 정리되어 매우 얇은 획에서는 손실 위험이 있음.
- 짝수 커널 에러 메시지:
 - OpenCV(4.11.0): (-215:Assertion failed) ksize.width > 0 && ksize.width % 2 == 1 && ksize.height > 0 && ksize.height % 2 == 1 in function 'cv::createGaussianKernels'

조건	Gaussian	Median	Gaussian 3 대비 평균 차이	이진 픽셀 수	작은 컴포넌트 수	판단
----	----------	--------	---------------------	---------	-----------	----

조건	Gaussian	Median	Gaussian 3 대비 평균 차이	이진 픽셀 수	작은 컴포넌트 수	판단
gaussian-3	3	0	0.0000	14031	12	기본 후보: 얇은 연필선 보존에 가장 보수적
gaussian-5	5	0	0.2836	15188	3	미세 점 감소, 선 확장/블러 증가
gaussian-7	7	0	0.6250	16833	1	과도한 스무딩 후보, 얇은 획 손실 위험
gaussian-3-median-3	3	3	0.1063	14460	4	점 노이즈 감소 효과, 매우 얇은 획에는 주의

Stage 03a: Grayscale 변환

입력	출력
	

- 동작: BGR → GRAY (`cv::COLOR_BGR2GRAY`)

[03a] Why

이진화와 형태학 연산은 단채널(1ch) 강도값만 필요. 컬러 정보는 ColorValidationFilter에서 이미 사전 검증됨.

OpenCV의 BGR2GRAY는 ITU-R BT.709 가중 평균 사용: $Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$ (인간 눈의 녹색 민감도가 가장 높다는 광생리학 기반)

[03a] 직접 테스트

1. 디버거에서 `gray.type()`이 `CV_8UC1(=0)`인지 확인 → 채널이 1로 줄었음을 직접 확인
2. `BGR2GRAY` 대신 단순 채널 추출(`cv::extractChannel(input, gray, 1)` — G채널만)으로 바꿔보고 결과 차이 관찰

3. 이 1채널 이미지를 그대로 `adaptiveThreshold` 다음 단계로 넘기면 정상 동작, 3채널 이미지를 넘기면 어떤 에러?

 내 관찰 결과:

- `gray.type()` 디버그 값:
 - BGR2GRAY 결과: `gray.type() = 0`, `gray.channels() = 1` → `CV_8UC1` 확인.
 - G채널 추출 결과도 `type() = 0`, `channels() = 1`로 `adaptiveThreshold` 입력 조건은 만족함.
- G채널 단순 추출 vs 가중 평균 차이:
 - 이번 입력은 거의 흑백 스캔/사진이라 차이가 매우 작음.
 - 평균 절대 차이(`meanAbsDiff`)는 `0.0031`, 서로 다른 픽셀 수는 `1798 / 589824`.
 - 다음 이진화 결과도 BGR2GRAY는 `14031px`, G채널은 `14035px`로 전경 픽셀 차이가 `4px`에 그침.
- 단, 일반 컬러 입력에서는 G채널만 쓰면 R/B 정보가 버려지므로, 색상 편향 없이 명도를 보존하는 BGR2GRAY가 기본값으로 더 안전함.
- 3채널을 이진화에 넣었을 때 에러 메시지:
 - `OpenCV(4.11.0): (-215:Assertion failed) src.type() == CV_8UC1 in function 'cv::adaptiveThreshold'`

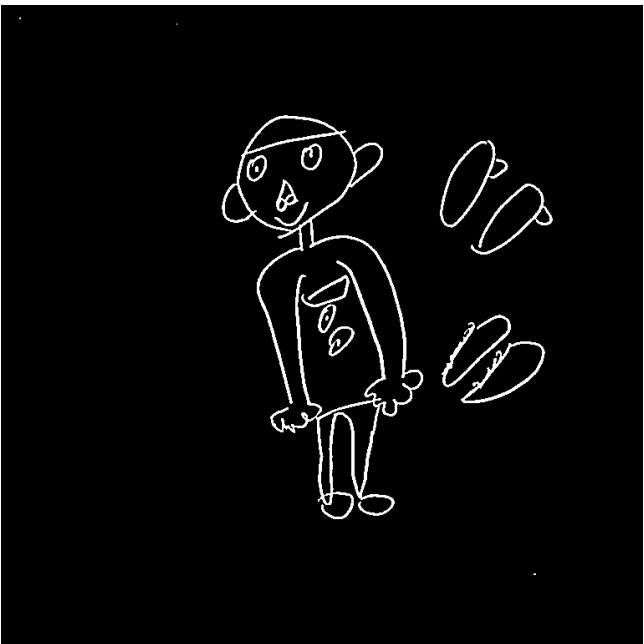
항목	값
BGR2GRAY type	0 (CV_8UC1)
BGR2GRAY channels	1
G채널 type	0 (CV_8UC1)
G채널 channels	1
BGR2GRAY vs G채널 평균 절대 차이	0.0031
BGR2GRAY 이진화 전경 픽셀	14031
G채널 이진화 전경 픽셀	14035
이진화 결과 차이 픽셀	14

Stage 03b: 적응형 이진화

입력	출력
----	----

입력

출력



- **파라미터** (ADR 권장): `AdaptiveThreshold`, `THRESH_BINARY_INV`, `blockSize=7`, `C=3`
- **동작**: 국소 영역별 임계값으로 이진화. 선 → 흰색(255), 배경 → 검은색(0)

△ 시각화에서 `blockSize=11`, `C=2`로 표기된 부분은 ADR 실측 권장값(7, 3)으로 통일 필요.

[03b] Why — 왜 Otsu가 아닌가?

전역 임계값(Otsu) vs 적응형(Adaptive):

알고리즘	동작	약점
Otsu	전체 히스토그램에서 최적 임계값 1개	조명 그라디언트가 있으면 밝은 쪽 선이 배경으로 분류됨
Adaptive	7×7 블록 단위로 지역 평균 계산	조명 불균일에 강건 (코드 채택)

`THRESH_BINARY_INV`인 이유: 이후 ROI 감지(`findContours`)는 흰색을 전경으로 본다. 인물선이 전경이어야 하므로 반전.

blockSize 파라미터 (ADR 실측):

blockSize	PSNR	SSIM	판정
7	19.6	0.9382	선택: 최고 SSIM, 원본 구조 보존
11	19.2	0.9355	미세 선 손실
15	18.9	0.9327	과도한 평균화

- `blockSize`가 클수록 지역 평균 영역이 넓어져 얇은 선이 "배경 평균과 비슷한 밝기"로 뭉개짐
- 아동 인물화의 선 간격이 보통 10~30px → 7px 블록이 적절

C 파라미터 (ADR 실측):

C	PSNR	SSIM	판정
1	19.0	0.9318	노이즈 픽셀이 선으로 잔류
2	19.2	0.9355	중간
3	19.3	0.9375	선택: 배경 처리 최적

- C는 지역 평균에서 빼는 상수. 클수록 더 많은 픽셀이 흰색(배경)으로 분류
- C=1: 노이즈가 선으로 잔류 / C=5+: 연한 연필선이 배경으로 사라짐

코드: [binarize_filter.cpp](#)

[03b] 직접 테스트

1. **Adaptive vs Otsu 비교**: `BinarizeFilter`의 `adaptiveThreshold`를 다음으로 교체

```
cv::threshold(blurred, binary, 0, 255, cv::THRESH_BINARY_INV |
cv::THRESH_OTSU);
```

조명이 불균일한 그림(한쪽이 어두운 사진)을 입력해 결과 비교

2. **blockSize 실험**: 7 / 11 / 15 / 21로 바꾸며 출력 비교 → 얇은 선이 어느 값부터 사라지는가?
3. **C 실험**: 1 / 3 / 5 / 10으로 바꾸며 → 배경 노이즈 vs 선 소실 트레이드오프 관찰

📄 내 관찰 결과:

- Otsu 적용 시 어두운 영역의 선 상태:
 - 같은 입력에 인위적 좌우 조명 그라디언트를 준 뒤 비교함.
 - Adaptive(b7,c3)는 전경 12304px로 baseline 12518px와 유사했지만, Otsu는 전경 306389px로 배경까지 대량 전경화됨.
 - Adaptive vs Otsu 차이 픽셀은 293969px로, 불균일 조명에서는 Otsu가 어두운 배경을 선처럼 오분류하는 경향이 큼.
- blockSize=15에서 사라진 선:
 - blockSize가 7 → 11 → 15 → 21로 커질수록 전경 픽셀이 12518 → 13695 → 14668 → 15799로 증가함.
 - 이번 샘플에서는 선이 사라지기보다 주변 픽셀이 더 많이 선으로 붙어 선이 두꺼워지고 작은 컴포넌트도 3 → 6 → 7로 증가함.
 - blockSize=15부터 baseline 대비 차이 픽셀이 2202px로 커져, 얇은 획 주변의 국소 판정이 눈에 띄게 변함.
- C=10에서 사라진 선:
 - C=1은 작은 컴포넌트가 147개로 배경 노이즈/점이 많이 남음.
 - C=5는 전경 11733px로 baseline보다 줄고 작은 컴포넌트는 0개가 되어 노이즈 억제가 강해짐.
 - C=10은 전경 9839px로 더 줄어 연한 선 일부가 배경으로 빠질 위험이 커짐.

조건	전경 픽셀	컴포넌트 수	작은 컴포넌트 수	baseline 대비 차이 픽셀
baseline adaptive b7 c3	12518	19	3	0

조건	전경 픽셀	컴포넌트 수	작은 컴포넌트 수	baseline 대비 차이 픽셀
uneven adaptive b7 c3	12304	19	1	222
uneven Otsu	306389	9	3	293969
block 11 c3	13695	17	3	1207
block 15 c3	14668	20	6	2202
block 21 c3	15799	21	7	3343
block 7 c1	13598	164	147	1080
block 7 c5	11733	18	0	785
block 7 c10	9839	33	6	2679

Stage 03c: 형태학 연산 (MORPH_CLOSE)

입력	출력
	

- **파라미터:** MORPH_CLOSE, kernel 3×3 (선 연결용) / 5×5 (ROI 전처리용)
- **동작:** Dilation → Erosion 순서로 끊어진 선을 연결하고 작은 구멍 메움

[03c] Why — 왜 CLOSE이고, 왜 두 곳에서 커널 크기가 다른가?

연필화는 선이 불연속적으로 그어지는 경우가 많다. 끊어진 선은 컨투어 감지에서 하나의 덩어리로 인식되지 않아 ROI 계산이 부정확해진다.

MORPH_CLOSE vs MORPH_OPEN:

연산	순서	효과	인물화 적합성
CLOSE	팽창 → 침식	간격 메움 → 원래 두께 복원	☒ 끊어진 연필선 연결

연산	순서	효과	인물화 적합성
OPEN	침식 → 팽창	작은 객체 제거	✗ 얇은 연필선이 침식 단계에서 먼저 사라짐

커널 크기가 두 곳에서 다른 이유:

위치	커널	목적
EnhanceContours (선 연결)	3×3	1~2px 미세한 끊김만 메움 — 더 큰 커널은 얇은 선 자체를 굵게 만들어 디테일 손실
GetContentROI (ROI 전 처리)	5×5	인물 전체 실루엣을 하나의 덩어리로 합치기 — 정밀도보다 연결성 우선

코드: [morphology_filter.cpp](#)

[03c] 직접 테스트

- 1. MORPH_CLOSE를 MORPH_OPEN으로 교체 → 얇은 연필선이 사라지는 것 관찰
- 2. 커널 크기 3×3 → 7×7로 변경 → 선이 굵어지면서 디테일이 뭉개지는지 관찰
- 3. 벤치마크 실행: [morphology_benchmark.cpp](#) 생성된 `benchmark_morph_*.jpg` 비교

📄 내 관찰 결과:

- OPEN 적용 후 사라진 선의 위치:
 - MORPH_OPEN 3×3 적용 시 전경 픽셀이 12924 → 6718로 거의 절반 감소함.
 - 얼굴 머리선, 귀, 손가락, 몸통 내부의 짧고 얇은 선들이 끊기거나 사라짐.
 - 컴포넌트 수가 15 → 170으로 증가해, 선 연결보다는 선 파편화가 강하게 발생함.
- 커널 7×7에서 굵어진 정도:
 - MORPH_CLOSE 7×7은 전경 픽셀이 15341로 증가하고 컴포넌트 수는 9로 감소함.
 - 끊김은 더 잘 메우지만 얼굴 내부, 손, 옷 장식 주변의 선이 굵어지고 작은 간격이 뭉개짐.
 - 평균 처리 시간도 close-k3 12.7964ms → close-k7 28.9867ms로 약 2.27배 증가함.
- 벤치마크 결과 베스트 커널:
 - 이번 샘플 기준으로는 close-k3가 선 연결과 디테일 보존의 균형이 가장 좋음.
 - open-k3는 얇은 선 손실이 크고, close-k7은 연결성은 좋지만 디테일/시간 비용이 큼.

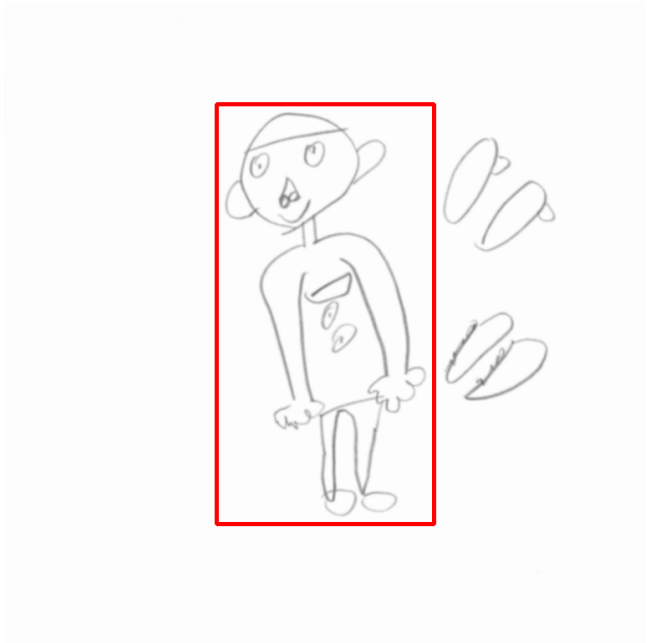
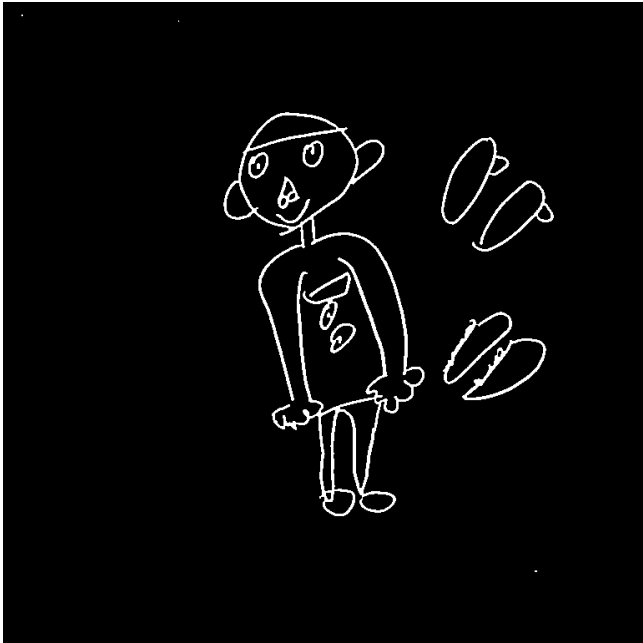
조건	전경 픽셀	컴포넌트 수	작은 컴포넌트 수	평균 처리 시간(ms)
close-k3	12924	15	3	12.7964
open-k3	6718	170	0	12.6535
close-k7	15341	9	3	28.9867

Stage 03d: Smart Crop (ROI 감지)

입력	출력
----	----

입력

출력



- 동작: `findContours` → 면적 상위 dominant 컨투어의 bounding box 추출 (+ padding 10px)
- 빨간 박스: 감지된 ROI 영역

[03d] Why

배경 여백을 포함한 채로 512×512에 맞추면 인물이 작아져 특징 추출 품질이 떨어진다. 메인 인물 영역만 잘라내서 리사이즈하면, 같은 512×512 안에 인물이 꽤 차게 들어가 EfficientNet이 더 풍부한 정보를 추론할 수 있다.

경계 조건 방어 (교재에는 없는 실전 코드):

방어	처리	위치
<code>contours.empty()</code>	전체 이미지 반환	L131
면적 < 0.1%	노이즈로 간주, 무시	L139
모든 rect 무효	fallback (전체)	L143
padding 후 경계 초과	<code>std::max/min</code> 클램핑	L154-157
ROI 유효성	<code>roi & imageRect</code>	L164

코드: [hybrid_preprocess_filter.cpp](#) — `GetContentROI()`

[03d] 직접 테스트

다음 엣지 케이스 입력으로 방어 코드 동작 검증:

1. 빈 이미지 (전체 흰색): 컨투어 없음 → 전체 이미지가 ROI로 반환되는가?
2. 점 하나만 있는 이미지: 면적 < 0.1% → 노이즈로 무시되는가?
3. 인물이 이미지 가장자리에 붙은 이미지: padding이 경계 초과 시 클램핑되는가?
4. 연필 얼룩만 있는 이미지: 모든 rect 무효 → fallback 동작?

📄 내 관찰 결과:

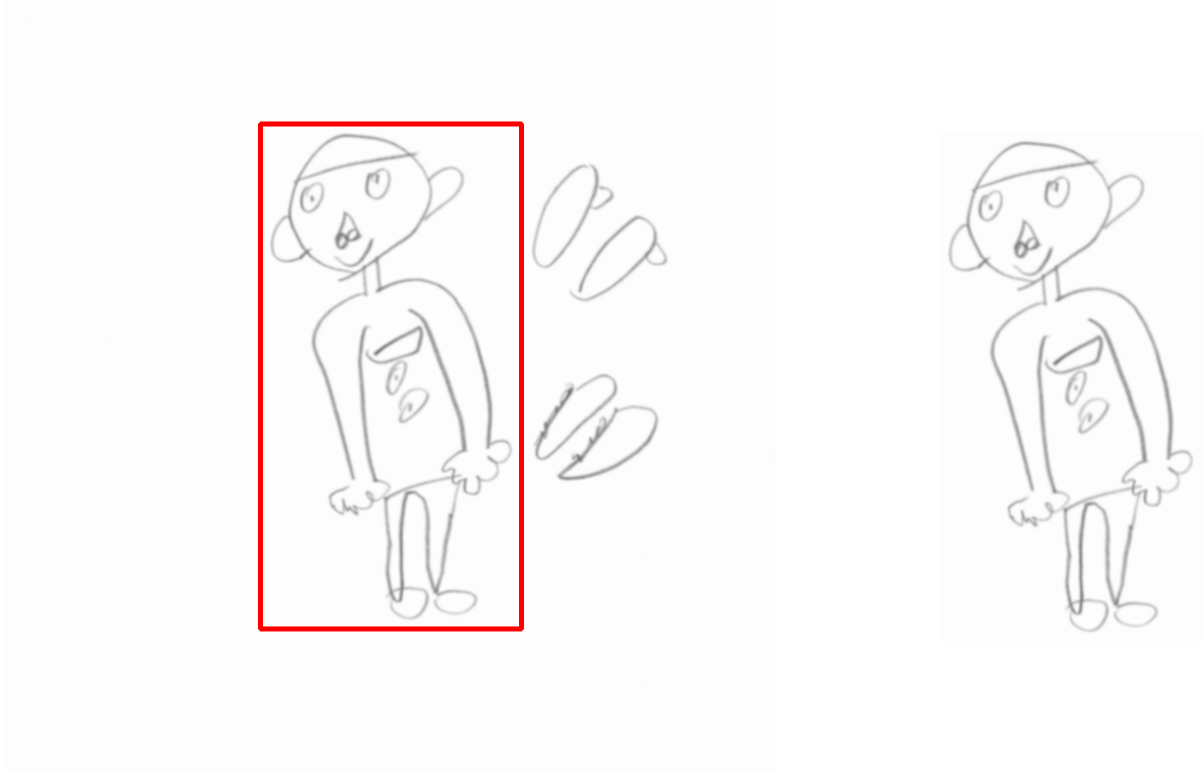
- 빈 이미지 ROI:
 - 전체 foreground가 없는 binary 입력에서는 contour가 없어 전체 이미지 256x256이 fallback ROI로 반환됨.
 - ROI: x=0, y=0, w=256, h=256.
- 가장자리 인물 → padding 후 좌표:
 - 좌측 가장자리에 붙은 synthetic 인물은 padding 후 x가 0으로 clamp됨.
 - ROI: x=0, y=38, w=100, h=180. 경계 밖으로 나가지 않고 정상 감지됨.
- 얼룩 이미지 fallback 결과:
 - 작은 점/얼룩만 있는 입력은 면적 0.1% 미만으로 무시되어 전체 이미지 fallback이 동작함.
 - single-dot, stains-only 모두 x=0, y=0, w=256, h=256.

케이스	x	y	width	height	결과
blank	0	0	256	256	fallback-full
single-dot	0	0	256	256	fallback-full
edge-touching	0	38	100	180	detected, padding clamp
stains-only	0	0	256	256	fallback-full

Stage 03e: Letterbox Resize (512×512)

입력 (ROI 영역)

출력



- 동작: ROI 영역을 비율 유지 리사이즈 후 **흰색(255)** 캔버스 중앙에 배치

[03e] Why — 왜 비율 유지, 왜 흰색 패딩?

비율 유지가 필요한 이유:

- AI 모델은 고정 입력 크기(512×512)를 요구
- 단순 stretch(비율 무시)를 하면 인물의 사지 비율이 왜곡 → HFD 문항 판정(팔 길이, 다리 비율 등)에 오류

흰색(255) vs 검은색(0) 패딩:

패딩 색	의미	문제
흰색 (255)	빈 도화지와 동일 픽셀값 → 도메인 일관성	☒ 채택
검은색 (0)	"정보 없음"이 아니라 어두운 픽셀로 해석	☒ EfficientNet이 어두운 영역으로 학습된 패턴을 잘 못 활성화

추가로, 다음 단계의 G채널(반전 이진화)에서도 흰색=배경이므로 Letterbox 패딩과 일관됨.

코드: [hybrid_preprocess_filter.cpp](#) — [PrepareLetterbox\(\)](#)

[03e] 직접 테스트

1. 흰색 캔버스(255)를 검은색(0)으로 변경 → 최종 3채널 이미지 시각적 차이
2. Letterbox를 비활성화하고 단순 `resize(512, 512)` 적용 → 인물 비율 왜곡 정도 관찰
3. ROI가 매우 가로로 긴 인물(전신화)에서 패딩 비율 측정

🔍 내 관찰 결과:

- 흰색 vs 검은색 패딩 시 G/B 채널 변화:
 - 흰색 패딩은 종이 배경과 같은 값이라 빈 도화지처럼 자연스럽게 이어짐.
 - 검은색 패딩은 좌우에 강한 검정 띠가 생겨, 모델 입력에서 실제 선/어두운 영역처럼 오해될 수 있음.
 - G/B 채널도 배경/패딩을 흰색 계열로 맞추는 설계라 흰색 패딩이 일관적임.
- 단순 `resize` 시 사지 비율 왜곡:
 - 실제 ROI는 260x503으로 세로가 긴 영역인데, 단순 512x512 `resize`는 가로 방향을 과도하게 늘림.
 - 얼굴/몸통/팔 폭이 넓어지고 전체 인물 비율이 눌러 HFD의 비율 관련 단서가 왜곡됨.
- 가로로 긴 ROI의 위/아래 패딩 픽셀 수:
 - `synthetic wide ROI(512x128)`를 `letterbox`하면 `placedY=192, placedHeight=128`.
 - 위/아래 패딩은 각각 192px로 대칭 배치됨.

케이스	ROI 크기	배치 영역	left	top	right	bottom
actual-white	260x503	264x511 @ (124,0)	124	0	124	1
wide-synthetic	512x128	512x128 @ (0,192)	0	192	0	192

Stage 04: 3채널 병합 결과

3채널 병합 후 각 채널을 분리해서 보면 각자 다른 정보를 담고 있음을 확인할 수 있다.

직관적 비유 — 같은 그림을 보는 세 명의 관찰자

같은 ROI 이미지를 **세 가지 다른 방식으로 가공**해서 3장의 회색 이미지를 만들고, 이를 R/G/B 채널 자리에 합쳐 한 장치럼 모델에 입력한다. 각 채널은 다음 질문에 답한다:

채널	답하는 질문	정보 유형	손실되는 것
R	"이 픽셀, 얼마나 진하게 그렸나?"	연속적 강도 (필압·음영)	— (원본 보존)
G	"이 픽셀, 선이야 종이야?"	이산적 형태 (윤곽)	강도, 굵기
B	"이 픽셀, 선의 중심부에 얼마나 깊이 있나?"	위상적 굵기	강도, 윤곽

세 채널이 일부러 **서로 다른 정보를 버리면서** 보완 관계를 이루는 게 핵심.

R 채널 — Grayscale (명도 / 필압)



- **가공 없음.** ROI 처리된 원본 그레이스케일을 흰색(255) 패딩 캔버스에 그대로 얹은 결과
- 픽셀값은 **0~255 모든 회색 톤**을 가짐 (예: 연한 연필=180, 진한 연필=50, 종이=240)
- 연필 압력(진하게 vs 연하게)이 명암으로 표현 → **연속적** 정보 보존

G 채널 — Inverted Binary (선 형태 / 윤곽)



이미지



구성 단계 (코드: [hybrid_preprocess_filter.cpp:55-59](#)):

1. `adaptiveThreshold(THRESH_BINARY_INV)` → 선=255(흰), 배경=0(검) 이진화
2. `bitwise_not` 으로 한 번 더 반전 → 선=0(검), 배경=255(흰)

→ 시각적 결과: 검은 선 + 흰 종이 형태의 깔끔한 이진 이미지

역할: "이 픽셀, 선인가 종이인가?"의 이진 판정만 담함.

- 1px 얇은 선이든 10px 굵은 선이든 G 채널에선 **모두 똑같이 검정** → 굵기는 알 수 없음
- 강도(농도)도 잃어버림 — G만 보면 진한 선과 연한 선의 차이 모름
- 대신 **형태/윤곽이 가장 깔끔하게 분리** → 모델이 공간적 패턴 학습에 유리

B 채널 — Distance Transform (선 굵기 / 위상)

이미지

이미지



구성 단계 (코드: [hybrid_preprocess_filter.cpp:62-65](#)):

1. `distanceTransform(DIST_L2)` — 각 선 픽셀에서 가장 가까운 배경 픽셀까지의 거리 계산
2. `normalize(0, 255)` — 거리값을 픽셀값 범위로 정규화 (선 중심=255, 가장자리=낮음)
3. `bitwise_not` 으로 반전 → 선 중심=0(어두움), 배경=255(밝음)

핵심 통찰 — 왜 거리 변환이 곧 굵기인가?

얇은 선 (2px):

□■□ ← ■는 선, □는 배경
→ 선 안 어디든 배경이 1px 옆 → 거리값 작음

굵은 선 (8px):

□■■■■■□
↑
한가운데 픽셀에서 배경까지 거리 = 4px → 거리값 큼

선이 굵을수록 그 안쪽의 거리값이 커진다 → 거리 = 굵기.

역할: "이 픽셀이 선의 중심부에 얼마나 깊이 있는가" → 굵기 / 선의 질.

- G가 못 잡는 굵기 정보를 명암(거리값을 시각화)으로 인코딩
- HFD 문항 중 "선의 질" 판정(굵기·일관성)에 직접 단서

잠깐, G와 B는 왜 둘 다 `bitwise_not`으로 반전되는가?

이건 개념적 역할과 무관한, 순수 기술적 처리다.

- 1차 처리 결과: G는 "선=흰, 배경=검", B는 "선 중심=흰, 배경=검"
- 만약 그대로 두면: 배경=0(검정), Letterbox 패딩=255(흰색) → **패딩 영역에 갑자기 밝은 띠가 생김** → 모델이 이를 강력한 신호로 오해

→ 반전 후: 배경=255 = 패딩=255 → **패딩과 배경이 동일한 흰색** → 모델이 패딩을 "빈 종이"로 자연스럽게 무시.

즉 반전은 "**Letterbox 패딩과 배경 색을 일치시키기 위한 마무리**"일 뿐, 채널의 의미는 변하지 않는다.

최종 3채널 병합 (AI 서버 입력)

이미지



- **R**: 명도 (필압) / **G**: 윤곽 (형태) / **B**: 거리 (굵기)
- 크기: **512×512×3**, dtype: uint8

[04] Why — 왜 단순 복제가 아닌가?

EfficientNet-B2는 ImageNet 사전학습 → 3채널 입력을 기대. 가장 단순한 방법은 `cvtColor(GRAY → BGR)`로 동일 채널을 3번 복제하는 것이지만, 그러면 **추가 채널이 새 정보를 전혀 제공하지 못한다.**

단순 복제 vs 보완적 채널:

채널	정보 유형	모델 학습 효과
단순 복제	동일 정보 × 3	중복, 학습 가치 0
R/G/B 다층 정보	연속 + 이산 + 위상	픽셀 강도, 공간 패턴, 위상 특징을 동시 학습

특히 B채널의 거리 변환은 HFD 문항 중 "**선의 질**" 판정(굵은 선/얇은 선/일관된 압력)에 직접적인 단서가 된다.

코드: [hybrid_preprocess_filter.cpp](#) — `ConstructChannels()` ADR: [hybrid_3channel_rationale.md](#)

[04] 직접 테스트

1. 각 채널을 `cv::imwrite`로 개별 저장 → 시각적으로 정보가 다른지 확인
2. 3채널을 단순 복제(`cvtColor GRAY2BGR`)로 변경 → AI 서버 추론 정확도 비교
3. B채널만 살펴보고 선 중심부의 "히트맵" 효과 확인 (굵은 선 영역의 픽셀값이 큰가?)
4. R/G/B 각각을 0으로 마스킹한 입력으로 추론 → 어느 채널이 가장 큰 영향을 주는가?

내 관찰 결과:

- R/G/B 채널 시각적 차이:
 - R 채널은 연필선 농담이 남는 연속 명도 이미지임.
 - G 채널은 선/배경 판정을 담는 이진 윤곽 이미지로, 선 강도 정보는 사라지고 형태만 남음.
 - B 채널은 distance transform 기반이라 선 중심부/굵기 정보가 회색 톤으로 나타남.
- 단순 복제 vs 다층 정보 → 추론 결과 차이:
 - 로컬 전처리 테스트에서는 추론 모델/정답 라벨을 사용하지 않아 정확도 비교는 실행하지 않음.
 - 대신 단순 복제 입력(`simple-gray-replicated`)과 다층 3채널 입력(`final-3ch`)을 생성해 후속 AI 서버 비교 입력으로 저장함.
- B채널 히트맵에서 가장 밝은 부분의 위치:
 - 현재 구현은 distance map을 정규화한 뒤 반전하므로, 시각 이미지에서는 선 중심부가 더 어둡게 나타남.
 - 반전 전 distance transform의 최대값은 3.5969이고 위치는 x=272, y=370으로 측정됨.
- 채널 마스킹 실험 → 가장 영향력 큰 채널:
 - R/G/B 각각을 0으로 마스킹한 입력 이미지는 생성했지만, 영향력 판단은 AI 서버 추론 결과가 필요해 이번 로컬 전처리 테스트에서는 보류함.
 - 생성 파일: `04-mask-r-zero.png`, `04-mask-g-zero.png`, `04-mask-b-zero.png`.

항목	값
R 채널 non-white 픽셀	134904
G 채널 stroke 픽셀	9813
distance transform 최대값	3.5969
distance transform 최대 위치 x	272

항목	값
distance transform 최대 위치 y	370

정리 — 단계별 입출력 요약

Stage	필터	입력	출력	핵심 이유	검증 가능한 ADR 수치
00	—	원본	BGR 임의 크기	—	—
01	ResizeFilter	BGR 임의	BGR ≤768px	47% 레이턴시 감소	183ms → 97ms
02	DenoiseFilter	BGR 768	BGR 768 (smooth)	이진화 노이즈 방지	gaussianSize=3, SSIM 0.9883
03a	cvtColor	BGR 768	Gray 768	단채널 연산 준비	—
03b	BinarizeFilter	Gray	Binary (INV)	선/배경 분리	blockSize=7 SSIM 0.9382, C=3 SSIM 0.9375
03c	MorphologyFilter	Binary	Binary (closed)	끊어진 선 연결	커널 3×3 (선 연결) / 5×5 (ROI)
03d	GetContentROI	Binary	ROI rect	인물 영역 추출	5단계 경계 조건 방어
03e	Letterbox	Gray ROI	Gray 512	비율 보존 + 흰색 패딩	도메인 일관성
04	ConstructChannels	Gray+Binary	RGB 512×512×3	다채널 정보 통합	연속+이산+위상

복습 워크플로우 제안

- 1. **Week 1 — L1 데이터 구조 감각**: 각 Stage의 "직접 테스트" 1번 항목만 모두 실행 → 채널 수, 픽셀값 범위 확인
- 2. **Week 2 — L2 변환 로직**: 각 Stage의 비교 실험(다른 후보값 적용) → 결과 이미지 비교, 본 문서 "내 관찰 결과"란 채우기
- 3. **Week 3 — L3 제약/시스템**: Smart Crop 엣지 케이스 + 벤치마크 직접 빌드/실행 + 새 필터 추가 도전

상세 학습 절차: docs/learning/preprocess_server_learning_guide.md